

Quantized GPT-2 Inference with Exact WebAssembly Proofs: Arithmetic, Cached Execution, and Numerical Evidence

Jamie Stephens*

24 September 2026

Abstract

Quantization changes a language model’s arithmetic, storage, and output, while deployment adds obligations about compiled instructions and retained state. We extend LeanExe’s GPT-2 124M example with signed eight-bit weights and projection inputs, 32-bit integer accumulation, and binary32 operations elsewhere. A Lean-checked theorem connects the exact 28,315-byte WebAssembly binary to the specified quantized cached-inference algorithm, including validation, termination, allocation, and buffer release for sessions of at most 128 tokens. The specification fixes scale selection, nearest-even rounding, saturation, and the order of all rescaling operations. An initial scale per activation row caused severe output degradation. Scales for consecutive groups of 64 coordinates improved greedy agreement with the binary32 implementation from 87 to 120 of 128 retained prefixes. The deployed grouped model matches its independent reference bit for bit on 6,432,896 logits, 128 complete caches, and nine completions. Its checkpoint occupies 74.3% less storage. A controlled resident-session benchmark measures a 3.59-fold median speedup and a 34.7% reduction in warm WebAssembly memory. Across 302 evaluated positions, including prefixes of six held-out prompts, greedy choices agree at 263 positions. Separate certificates derived from observed logit pairs establish 232 individual choices after removing a common offset. Conditional numerical theorems propagate quantization and floating-point errors through the complete cached recurrence, but their evaluated forward bounds certify zero token choices. We give the theorem boundary, data identities, measurement procedures, and remaining assumptions, including native checker execution and the identity of captured operands with source intermediates.

Contents

1	Verified quantized cached inference	3
1.1	Relationship to quantization and verification work	3
2	Quantization and arithmetic order	4
2.1	Scale selection and coefficient encoding	4
2.2	Grouped projection	4
2.3	Accumulator range and conversion	5
2.4	Model format and validation	5
3	Exact binary and session proof	6
3.1	Statement for the distributed module	6
3.2	Scalar operations, tensors, and proof reuse	7
3.3	Cache ownership and allocation sufficiency	7

*Morphism (<https://morphism.com>). Contact: js@morphism.com.

4	Checkpoint export correspondence	8
5	Accuracy diagnosis and grouped evaluation	9
5.1	Reference agreement and original failure	9
5.2	Group64 experiment and vocabulary control	9
5.3	Generated text and held-out inputs	10
6	Storage, runtime, and memory measurements	11
6.1	Projection costs	12
7	Conditional numerical bounds and token certificates	13
7.1	Projection error with rounded rescaling	13
7.2	Nonlinear operations and cached recurrence	13
7.3	Captured range evidence and forward failure	14
7.4	Certificates from observed logit pairs	15
8	Trust assumptions and remaining work	16
A	Source, artifact, and evidence identities	16
A.1	Claim-to-declaration map	17
A.2	Measurement records	18
A.3	Reproduction procedure	19

1 Verified quantized cached inference

A Lean-checked theorem connects the exact 28,315-byte WebAssembly binary to the specified quantized cached-inference algorithm, including validation, termination, allocation, and buffer release for sessions of at most 128 tokens. The model has GPT-2’s 124,439,808 parameters, twelve transformer blocks, hidden width 768, twelve attention heads, feed-forward width 3,072, and vocabulary size 50,257. The checkpoint stores 1,024 position embeddings. The deployed recurrence accepts positions zero through 127. We use the existing checkpoint, tokenizer, prompts, serial binary32 implementation, and reference harness from the earlier cached-inference work [4, 1, 5].

The new computation quantizes learned projection weights and their activation inputs to integers in $[-127, 127]$. Each group of 64 activation coordinates has its own scale. Every output channel has one weight scale across its input width. Integer products accumulate in 32-bit words, and each group sum converts to binary32 before rescaling and addition. Token embeddings share the quantized vocabulary matrix. Position embeddings, normalization parameters, biases, attention scores and values, cached keys and values, GELU, softmax, residual additions, and output logits retain binary32 arithmetic.

The execution proof establishes exact agreement with the quantized algorithm for the frozen binary. Experiments measure the algorithm’s agreement with its independent reference and its differences from the binary32 model. Numerical theorems bound those differences under explicit assumptions. The evaluated forward bounds are too large to establish a token choice. Certificates constructed from already observed logit pairs establish some individual choices. These conclusions have different premises and evidence, which Sections 3, 5, and 7 specify.

The report fixes the implementation and evidence at LeanExe revision `c655d35b5011c1703dfd22bcceaec4e5bee17088`. Quantized command-line inference is opt-in through `tools/gpt2--quantized`. The binary32 path remains the default. The focused quantized source and exact-binary package checks have passed. Repository-wide release work remains open at this revision: current limit/market matcher proof repairs, their preserved artifact packages, aggregate source and artifact checks, the complete execution and conformance tests, and a matching clean-checkout receipt. The reported GPT results therefore identify a checked component and recorded experiments within an unfinished repository release.

1.1 Relationship to quantization and verification work

Integer quantization with explicit scale and rounding rules follows established inference practice. Jacob et al. describe integer-only inference and training designed to preserve accuracy [3]. The present implementation uses post-training quantization and binary32 rescaling and nonlinear operations. Its deployed mathematical object is a mixed integer/binary32 recurrence with a fixed evaluation order.

Activation outliers have motivated specialized transformer quantizers. `LLM.int8()` combines vector-wise quantization with a separate higher-precision computation for outlier dimensions [2]. SmoothQuant transfers activation scaling difficulty into weights through an equivalent transformation before quantization [7]. Our experiment instead changes the activation scale granularity to 64-coordinate groups while preserving eight-bit products and the exported coefficient payload. The comparison in Section 5 measures this particular change on GPT-2. The benchmark compares two LeanExe WebAssembly implementations on one host, so it supplies no runtime comparison with those systems.

QEBVerif uses differential reachability analysis and mixed-integer linear programming to verify bounds between neural networks and their quantized counterparts [8]. Our numerical work proves compositional error formulas in Lean for the integer and binary32 operations of a cached transformer.

The binary proof additionally covers the deployed instructions and memory operations. The present forward evaluator’s failure to establish useful margins is a concrete limitation of its error formulas. The earlier LeanExe reports supply the binary32 session, floating-point semantics, and proof organization reused here [1, 5].

2 Quantization and arithmetic order

2.1 Scale selection and coefficient encoding

Let $\text{fl}_{32}(x)$ denote the binary32 result of the specified rounded operation, and let $\text{rne}(x)$ denote rounding a represented value to the nearest integer with ties to even. For a finite vector $x = (x_0, \dots, x_{d-1})$, the scale rule is

$$m = \max_i |x_i|, \quad s(x) = \begin{cases} 1, & m = 0, \\ \max\{2^{-126}, \text{fl}_{32}(m/127)\}, & m > 0. \end{cases} \quad (1)$$

The implementation compares the magnitude bits of finite binary32 words to obtain m . The floor is the smallest positive normal binary32 value. It prevents a positive scale from becoming zero or subnormal. The all-zero vector has scale one and all-zero coefficients.

The signed coefficient is

$$q_s(x_i) = \text{rne}\left(\text{clip}_{[-127,127]}(\text{fl}_{32}(x_i/s))\right). \quad (2)$$

Division rounds before clipping, and clipping precedes nearest-even rounding. The result is converted to a signed 32-bit integer and stored as its low byte. Thus -127 has byte representation `0x81`, zero has `0x00`, and 127 has `0x7f`. The validator rejects `0x80`, the encoding of -128 . Zero point zero is implicit. Both signs of an input zero produce integer zero. The definitions `rowScale`, `quantizeValue`, and `quantizeRows` implement these rules in the projection source.

Each learned projection stores coefficients in output-major order: all input coordinates for output channel j occupy one contiguous row. Its scale s_j^w comes from Equation (1) applied to that complete weight row. Token embedding row j uses the same rule and the same coefficient row later used to compute vocabulary logit j . Learned projection biases and normalization parameters retain their original binary32 words.

2.2 Grouped projection

Let the input width be $d = 64G$. GPT-2 uses $G = 12$ for width 768 and $G = 48$ for width 3,072. For input row r , group g contains coordinates $64g$ through $64g + 63$. Its scale is $s_{r,g}^x$, and its coefficients are $q_{r,g,i}^x$. With output-major weight coefficients $q_{j,g,i}^w$, the projection computes

$$a_{r,j,g} = \sum_{i=0}^{63} q_{r,g,i}^x q_{j,g,i}^w, \quad (3)$$

$$u_{r,j,g} = \text{fl}_{32}\left(\text{f32}(a_{r,j,g}) \text{fl}_{32}(s_{r,g}^x s_j^w)\right), \quad (4)$$

$$t_0 = +0, \quad t_{g+1} = \text{fl}_{32}(t_g + u_{r,j,g}), \quad (5)$$

$$y_{r,j} = \begin{cases} \text{fl}_{32}(t_G + b_j), & \text{with bias,} \\ t_G, & \text{without bias.} \end{cases} \quad (6)$$

The inner scale product in Equation (4) rounds first. The accumulator converts to binary32 exactly under the range bound below. A second multiplication rounds the rescaled partial sum. Equation (5) adds groups in increasing index order, starting from positive zero. Bias is added once after the last group. The vocabulary projection uses the bias-free case. The grouped source definition fixes this order, including each binary32 rounding point.

The original candidate used one activation scale and one integer dot product for the complete input row, followed by one rescaling. Grouping therefore changes both the quantizer and the floating-point accumulation order. Its measured improvement cannot be attributed to scale granularity alone while assuming identical arithmetic elsewhere. The weight coefficients, per-output weight scales, and quantized embedding values remain unchanged between the two schemes.

2.3 Accumulator range and conversion

Every permitted coefficient has magnitude at most 127, so every integer product has magnitude at most $127^2 = 16,129$. For every prefix of k products,

$$\left| \sum_{i < k} q_i^x q_i^w \right| \leq k \, 16,129. \quad (7)$$

For $k \leq 64$, the bound is $1,032,256 < 2^{20}$. Products and all partial sums fit the signed 32-bit range, and the final integer converts to binary32 exactly because its magnitude is less than 2^{24} . For the original complete-row kernel, $k \leq 3,072$ gives $49,548,288 < 2^{31}$. That larger bound still proves accumulation safety, while a complete-row accumulator can require rounding on conversion to binary32.

The implementation stores products and sums as `UInt32` words. The checked `dot_exact` theorem identifies the signed decoding of the resulting word with the mathematical integer sum. Its companion `dot_prefix_range` proves Equation (7). These results use the exclusion of byte `0x80` and the width bound to justify the implementation’s word arithmetic. The grouped numerical proof uses the sharper 64-term bound and the checked integer-to-binary32 conversion theorem.

2.4 Model format and validation

The version-one checkpoint occupies 127,695,972 bytes: a 32-byte header and a 127,695,940-byte payload. Words are little-endian, and tensor starts are four-byte aligned. The header contains the magic string `LXQGPT2` followed by zero, format version one, header and payload lengths, scheme identifier two, context limit 128, and a zero reserved word. Scheme one identifies the preserved complete-row candidate. Scheme two identifies Equations (3)–(6). The tensor payload is identical between the schemes, while the changed header produces a different whole-file hash.

The payload contains 123,532,032 coefficient bytes, 133,201 binary32 scales, and 907,776 retained binary32 words. The shared token/vocabulary matrix contributes 38,597,376 coefficient bytes and 50,257 scales. Twelve transformer blocks follow the position embeddings. The final normalization parameters end the file. The tensor manifest gives all offsets, dimensions, and lengths, and the format specification states the public API.

Initialization checks the complete header, excludes every forbidden coefficient, requires every scale to be finite and at least 2^{-126} , and checks every retained binary32 parameter for finiteness. Each token call checks the header again, token and position bounds, cache length, and cache finiteness. The status codes are zero for success, one for format rejection, two for model-parameter rejection during validation, three for token/cache rejection, and four for numerical failure. Failure

returns empty output buffers. A finite validated checkpoint can still cause a numerical failure during inference. Validation establishes its represented shape and permitted values, while the execution theorem covers the resulting success or failure path.

3 Exact binary and session proof

3.1 Statement for the distributed module

The execution theorem proves the behavior of a fixed byte sequence. Lean embeds the 28,315 bytes, checks their decoding and validation, and proves equality between the decoded module’s Talos translation and the execution model used by the behavioral proofs. The package checker compares the external `program.wasm` file with those embedded bytes. The SHA-256 digest identifies the file for retrieval and measurement. The byte comparison and decoding proof establish its formal connection to the execution model.

Write B for the embedded bytes, R for a decoded raw module, V for its validated representation, and $T(V)$ for its translation to the Talos execution semantics. The theorem `Project.Gpt2QuantizedCached.Artifact.artifact_gpt2_128_exact` proves

$$\begin{aligned} \exists R, V : \quad & \text{decode}(B) = \text{ok}(R) \\ & \wedge \text{Grammar. Encodes}(B, R) \\ & \wedge \text{validate}(R) = \text{ok}(V) \\ & \wedge \text{CoreValid}(R) \\ & \wedge \text{ExactSpecFor}(T(V)). \end{aligned} \tag{8}$$

The translation proof checks all 66 function bodies and the complete module equality. `Grammar.Encodes` is the declarative binary-encoding relation. `CoreValid` is the checked verifier profile’s validity property. `ExactSpecFor`, defined in the complete specification, states the reset, allocation, loading, validation, and session result below.

Theorem 1 (Quantized session execution). *For every byte array W of length 127,695,972 and list of 32-bit token words τ of length at most 128, the specified session over $T(V)$ terminates. It resets the instance, allocates the weight buffer, writes W , and returns the source model validator’s status. If validation succeeds, each requested token call returns the exact status, cache bytes, and logit bytes of the quantized Lean recurrence. A nonzero token status ends the trace. The specified releases terminate, and memory stays within the module’s 65,536-page limit.*

The theorem quantifies over all weight arrays of the required length. It includes arrays rejected by validation and successful validation followed by a numerical failure. Successful inference for the pinned pretrained checkpoint is established by the recorded runs. A successful step produces 50,257 binary32 logits and appends 73,728 cache bytes. The source trace follows the model’s own status and stops on failure, so termination covers failure cleanup as well as successful traversal.

The proof uses the Lean model as the behavioral specification and proves the connection from the emitted bytes. It therefore needs no general compiler-correctness theorem. Compiler output, annotations, and cached syntax are checked proof inputs. A changed binary requires a new exact-byte package and proof. The package’s axiom audit reports `propext`, `Classical.choice`, and `Quot.sound`. These are the standard Lean axioms accepted for this development.

3.2 Scalar operations, tensors, and proof reuse

The quantized loop proof composes byte decoding, signed extension, integer multiplication and addition, binary32 division, nearest-even rounding, integer conversion, rescaling, ordered addition, and packed output stores. The binary32 operations use the existing integer-defined arithmetic semantics. The WebAssembly numerical specification supplies the deployment meaning of these instructions, including rounding behavior [6]. Wasmtime runs the retained binaries with arithmetic NaN canonicalization enabled to match the chosen deterministic semantics.

The model retains the earlier binary32 implementations of normalization, cached attention, residual addition, and GELU. Checked function-region equalities transfer their execution and memory theorems into the quantized module after function and type index renaming. The binary32 region proof covers twenty helpers, and the projection region proof covers twelve projection and runtime helpers. The equality checks justify reuse at the instruction level. Shared source names or apparent similarity alone provide no such transfer.

The transformer-block proof follows two normalizations, four quantized projections, attention, two residual additions, GELU, and key/value extraction. It establishes exact status and bytes, protected-input preservation, returned ownership, and cleanup along each finite-value branch. The twelve-block traversal carries those facts through the hidden state and accumulated cache updates. A final normalization and vocabulary projection produce the logits. The public step theorem also checks token masking at the 64-bit calling interface, position arithmetic, input rejection, and output finiteness guards.

3.3 Cache ownership and allocation sufficiency

At position p , the cache contains $12 \cdot 2 \cdot 768p = 18,432p$ binary32 words, or $73,728p$ bytes. Attention reads the appropriate prior keys and values and the current QKV projection. Each block returns 1,536 new words, and the traversal collects the twelve block updates before appending them to the prior cache. The layout and copying order are part of the source recurrence and the byte-level execution proof.

The caller retains the old cache until the token call returns. A successful call supplies a replacement cache and a logit buffer. After consuming the outputs, the specified host sequence releases the old cache and then the logit buffer. A failed call ends the session and releases the remaining cache and weights. Shutdown also releases the final cache and weights after the last successful requested token. Ownership assertions track live packed objects and disjoint protected buffers through allocator reuse and reference-count changes.

The allocation proof establishes sufficient capacity before each allocation. A block’s heap-top growth is at most 96 KiB under the proved position bound. The complete cached hidden function has growth bounded by the incoming cache byte length plus 1,736,456 bytes. The public token step increases the heap top by at most the incoming cache byte length plus 1,942,544 bytes. The session invariant reserves an initial 128 MiB and 16 MiB for each token:

$$\text{heapTop}(p) \leq 134,217,728 + 16,777,216p, \quad p \leq 128. \quad (9)$$

This reservation fits the 32-bit linear-memory limit. The proof also tracks the memory-page bound and fixed capacity of 65,536 pages. Equation (9) is a sufficient formal reservation, while the measured memory capacities in Section 6 describe one allocator execution. Neither quantity is process resident memory.

The formal session uses a specified host byte-write operation and a specified sequence of calls. Its implementation boundary includes the native host’s corresponding copies, argument encoding, calls,

output reads, and releases. Wasmtime, its compiler, the operating system, and hardware execute those operations outside the Lean kernel. Section 8 states the remaining runtime assumptions.

4 Checkpoint export correspondence

The exporter reads the pinned `openai-community/gpt2` checkpoint and writes the fixed quantized layout. Its revision, source-file digest, tensor manifest, original binary32 weight digest, and quantized weight digest are recorded. The learned block matrices change from the source layout to output-major coefficient rows. The shared token embedding and vocabulary matrix retains its token-row orientation. A mistaken transpose would change inference while leaving the byte count valid, so the export relation states the source index corresponding to every target row and coordinate.

The checked export predicate compares the binary32 and quantized files. For each of 49 projection matrices, it checks the specified row scale and every coefficient from Equations (1) and (2). For each retained tensor, it checks finiteness and equality of the binary32 words. The complete-file run accepted 123,532,032 coefficients, 133,201 scales, and 907,776 retained words. The first interpreted run timed out before completing its first matrix. The retained successful run evaluates the checked definitions through the pinned native Lean compiler and runtime. The export record preserves both outcomes.

The theorem `Export.projection_checked` derives the per-row check from the whole-file predicate. `Export.retained_checked` supplies retained-word equality and finiteness. `QuantizedExport.reconstruction` connects accepted quantizer rows to a real-valued reconstruction bound. The proofs of these implications check in the Lean kernel. The successful evaluation over all checkpoint bytes uses the native execution boundary described above. The checkpoint’s identity with the downloaded training artifact uses the recorded file hashes and exporter inputs.

For a finite input x , positive finite scale s , and rounded quotient $r = \text{fl}_{32}(x/s)$, the scalar theorem gives

$$|sq_s(x) - x| \leq s \left(\frac{1}{2} + \max(0, |r| - 127) + \varepsilon_{\text{div}} \right). \quad (10)$$

For a finite binary32 value y , define $M(y) = 2^{149}|y|$, a nonnegative integer that expresses its magnitude in units of 2^{-149} . The quotient-range premise for an integer exponent b with $24 \leq b \leq 275$ is

$$M(x)2^{149} \leq M(s)2^b, \quad \text{equivalently} \quad |x/s| \leq 2^{b-149}. \quad (11)$$

Under this premise, `F32DivisionBounds.div_real_error` bounds the division error by

$$\varepsilon_{\text{div}}(b) = \frac{2^{b-24}}{2^{149}} = 2^{b-173}. \quad (12)$$

Thus $b = 156$ asserts $|x/s| \leq 128$ and gives $\varepsilon_{\text{div}}(156) = 2^{-17}$. In Equation (10), the half-unit term comes from nearest-even rounding, the maximum term covers clipping, and $\varepsilon_{\text{div}}(b)$ covers the initial binary32 division. The quotient-range assumption is explicit in `QuantizedScalarError.reconstruction`. The proof accounts for rounded scale selection and finite division.

The retained activation records cover 233,580 groups and 14,949,120 coefficients at 229 evaluated positions. A native Lean checker verifies the same quantizer relation, finite inputs, positive finite scales, and quotient range. The largest observed scale is approximately 2.18501425, and the largest rounded quotient magnitude is $127 + 2^{-17}$. The checked exponent 156 bounds quotient rounding by

2^{-17} . Equation (10) then gives a component bound $s(1/2 + 2^{-16})$ for those records. Using their largest scale yields the exact uniform local bound

$$\frac{150,157,618,083}{137,438,953,472} \approx 1.09254046.$$

This quantity bounds activation reconstruction at the captured quantizers. Subsequent multiplication, rescaling, summation, and transformer operations add further error. The full-model bound treats those stages separately.

5 Accuracy diagnosis and grouped evaluation

5.1 Reference agreement and original failure

The first cached candidate reproduced its independent quantized reference on every logit and cache in the fixed 128-prefix trace. That implementation agreement coexisted with poor agreement with binary32 inference. Its greedy choice matched the binary32 model on 87 of 128 prefixes, and all nine retained completions differed within their first three generated tokens. The first sampled continuation contained the fragment “the first of the the Pisa, will to the that a the the”. Its binary32 counterpart described a village. The complete texts, token IDs, and shared sampling draws remain in the original completion record.

We diagnosed projection error before completing the full-model proof. A separate serial binary32 reference reproduced the frozen binary32 module’s logits bit for bit. For each of 49 projections at the first nine fixed prefixes, the diagnostic used that model’s input and compared activation-only reconstruction, weight-only reconstruction, and the integer projection against the original projection. Using the same input isolates a local operation’s error. It leaves the propagation of earlier quantization errors outside that comparison.

The vocabulary projection had the largest local RMS error at eight of nine prefixes. Its input maximum magnitude ranged from 53.4 to 198.5, while its median coordinate magnitude ranged from 0.115 to 0.312. The complete-row scale quantized 79.2–95.6% of its coordinates to zero. At prefix seven, activation quantization alone gave RMS output error 1.812, weight quantization alone gave 0.318, and the complete integer projection gave 1.839. Outliers also appeared within the transformer: block two’s expansion at the first prefix quantized 97.3% of its input coordinates to zero. Its activation-only RMS output error was 0.270, compared with 0.022 from weight reconstruction. The projection diagnostics support activation scaling as a substantial local error source.

5.2 Group64 experiment and vocabulary control

The experiment used independent scales for consecutive groups of 64 activation coordinates. It kept the coefficient payload and quantized embeddings fixed and adopted the ordered group-rescaling computation of Section 2. A control kept complete-row transformer projections but used binary32 vocabulary activations and serial binary32 multiplication of reconstructed quantized vocabulary weights. The control tests whether removing activation quantization at the last projection accounts for much of the original distribution error.

Table 1 reports greedy agreement and the KL divergence from the binary32 softmax distribution to each candidate. Each prefix supplies the same token sequence to both models, with each model maintaining its own cache. The fixed-trace record and prompt-prefix record retain every logit hash and per-prefix metric. The control’s improvement supports final-projection activation quantization

Input set	Arithmetic	Greedy matches	Median KL	Maximum KL
Fixed trace	Complete-row	87/128	0.506706	4.182229
	Group64	120/128	0.009023	0.251639
	Binary32 vocabulary control	120/128	0.003522	0.150031
Nine prompt prefixes	Complete-row	49/101	0.677073	3.736186
	Group64	85/101	0.066768	0.244526
	Binary32 vocabulary control	87/101	0.039378	0.240125

Table 1: Agreement with the frozen serial binary32 model. KL divergence is in nats. The binary32 vocabulary control retains quantized transformer projections and quantized vocabulary weights.

as a major contributor on these inputs. The experiment does not assign a causal share of final error to every transformer layer.

Raw logit differences contain a component common to all vocabulary entries. Such a shift preserves softmax probabilities and greedy order. With differences $\delta_j = z_j^q - z_j^f$ and their mean $\bar{\delta}$, the centered RMS is $\sqrt{\sum_j (\delta_j - \bar{\delta})^2 / 50,257}$. On the fixed trace, grouping reduced the maximum centered RMS from 1.862 to 1.052. Its largest absolute difference was 49.587 at prefix 57, where the mean shift was 43.054, centered RMS was 0.868, and KL divergence was 0.000507 nats. On prompt prefixes, the grouped maximum raw difference rose from the original 13.893 to 19.296, while maximum centered RMS fell from 1.502 to 0.866. Reporting both quantities separates a common offset from changed relative logits.

The compiled grouped binary matches all 6,432,896 reference logits and each of 128 complete caches bit for bit. Every logit vector also reproduces its grouped-experiment hash. The cached comparison therefore tests the complete deployed recurrence against the independently implemented serial quantized reference. The reference preserves the specified polynomial approximations and operation order. This test establishes agreement on its finite input set. The execution theorem establishes exact source agreement for all inputs satisfying its session setup.

5.3 Generated text and held-out inputs

The compiled grouped model reproduces all nine reference completions, including the two sampled cases using the same Lean PRNG draws. Table 2 gives the first generated position at which group64 and binary32 differ. The village sample begins with a village and a monster, while the science sample starts “to understand and test hypotheses”. Greedy completions still contain repetition and factual errors. The science, narrative, code, numerical, and explanatory prompts provide examples of changed behavior, but this small collection supplies no semantic-quality score.

The six prompts initially designated held-out in the first candidate’s manifest had already been evaluated when grouping was proposed. We therefore fixed another six prompts after the group64 implementation and before evaluating them. They cover narrative, exposition, dialogue, code, arithmetic prose, and matrix multiplication. Their complete input token lists and 32-token greedy requests appear in the held-out manifest. No quantizer tuning used their results.

On all 73 prefixes of those six prompts, greedy choices agree at 58 positions. The held-out logit record reports maximum absolute difference 18.1201. Their six generated continuations all diverge from binary32, at generated positions 7, 3, 4, 11, 6, and 1. Repetition occurs in both models. The quantized code continuation repeats a loop fragment, and its matrix explanation confuses row and column conditions. The complete held-out texts preserve these failures.

Across the fixed trace, nine original prompt-prefix sequences, and six later prompt-prefix

Prompt	Requested tokens	First difference
Once upon a time, in a small village	64	1
The purpose of science is	32	2
The capital of France is	16	13
A proof of the theorem begins with the following observation:	32	4
The train arrived at the station just before midnight.	32	4
<code>def sum_of_squares(values):</code> with initial total	32	7
zero		
The measurements were 0.125, 3.5, and 127.0. Their sum is	32	1
In the garden, the children found a small wooden box.	32	2
Q: Why does ice float on water? A:	32	1

Table 2: Group64 versus binary32 completion differences, with generated positions counted from one. The first two cases use top- $k = 40$, temperature 0.8, and seeds 42 and 7. The remaining cases use greedy decoding. Exact prompts, newlines, tokens, and draws appear in the completion record.

sequences, group64 and binary32 agree on 263 of 302 evaluated positions. These positions include repeated token prefixes across sets. The fixed 128-token trace also repeats a short phrase through most of its length. The observations characterize the retained tests and the six later prompts. They supply no estimate of corpus-wide accuracy, perplexity, factual reliability, or long-context behavior. The deployment decision remains opt-in quantized inference with binary32 as the default.

6 Storage, runtime, and memory measurements

The quantized checkpoint reduces storage by 74.3%. Retained binary32 tensors and scale words account for the difference from a fourfold reduction. Cache values remain binary32, so cache storage per token is unchanged.

The controlled full-model benchmark runs both frozen binaries on `appledev1`, aarch64 Linux, using Wasmtime 44.0.0 with canonical NaNs. It uses one warmup 128-prefix trace and three measured traces. It alternates the model order at each position and repetition. The execution scope gives the process one core of CPU quota, a 4 GiB memory-high threshold, 6 GiB memory maximum, and 1 GiB swap maximum, while the shared runner lock excludes concurrent Lean work. The record includes the host executable digest, CPU feature listing, Linux identity, source hashes, NumPy version 2.5.3, and PyTorch version 2.9.1+cpu.

Timing covers resident cached calls, logit transfer, and temporary-buffer release. Loading, validation, reference comparisons, and hashing occur outside the measured interval. Table 3 reports the retained warm-session measurements. Every repeated logit vector has the same hash. This is a comparison with LeanExe’s serial binary32 WebAssembly baseline on the recorded host and workload. PyTorch supplies a separate reference comparison outside the timing interval.

The complete one-trace cached comparison records different memory capacities: 737,673,216 bytes for group64 and 1,107,361,792 bytes for binary32 at position 128. Those figures describe the first trace’s terminal capacity. Table 3 describes the maximum after repeated warm traces, including allocator growth and reuse. WebAssembly linear memory retains allocated capacity until the instance closes, even after object release. Peak process resident memory also includes the native runtime and other process allocations.

After each successful token call and host cleanup, the quantized session retains two live allocations:

Measurement	Group64	Binary32
Checkpoint bytes	127,695,972	497,759,232
Median 128-prefix trace time (s)	27.287	97.936
Minimum measured trace time (s)	27.238	97.789
Maximum measured trace time (s)	27.313	97.947
Maximum warm linear-memory bytes	747,110,400	1,144,848,384
Peak process resident bytes	759,934,976	1,157,341,184

Table 3: One warmup trace followed by three measured 128-prefix traces with resident weights. The median time ratio is 3.59, and the warm linear-memory reduction is 34.7%. The small repetition count and single host limit the runtime conclusion to this experiment.

weights and current cache. By session close, the allocator has freed all 45,569 allocations made during the single 128-token session. The four-trace benchmark’s cumulative count is 182,273 allocations, all freed by close. The session tests also cover invalid headers, the wrong scheme, malformed model parameters, invalid tokens, invalid cache shapes or values, failures after allocation at three transformer depths, preservation of the preceding cache, reset, and close. These records test the native orchestration corresponding to the formal session.

6.1 Projection costs

The grouped projection experiment measures four checkpoint shapes on the original first-token binary32 inputs. It compares group64, complete-row quantization, and an output-major binary32 projection. All variants run under the same one-core resource scope. Each measurement follows a reference comparison and two warmup calls, then records seven timed calls. Timing includes activation quantization, projection, the host round trip, and output release. Weight loading and validation occur before timing.

Projection	Group64 (ms)	Row scale (ms)	Binary32 (ms)	Ratio
QKV, 768×2304	3.428	3.475	13.674	3.99
Expansion, 768×3072	5.311	5.365	21.465	4.04
Reduction, 3072×768	5.225	4.626	20.422	3.91
Vocabulary, 768×50257	77.938	78.903	303.692	3.90

Table 4: Median projection times and binary32/group64 ratio. Dimensions are input width by output width. These four local measurements use binary32-model inputs and therefore exclude accumulated full-model quantization error.

The reduction projection takes 12.9% longer with group64 than with a complete-row scale in this run. The other three medians differ by about 1%. Grouping adds activation scales and group-level binary32 work. Its checkpoint payload remains unchanged. Each grouped call allocates three buffers and releases all three after output consumption. Measured linear-memory capacity equals the complete-row projection in these four cases. The projection record includes all repetitions, output hashes, and allocation counts.

The grouped projection’s output matches its independent reference byte for byte for all four shapes. The reference uses 64-bit integer accumulation and checks the 1,032,256 bound, while the deployed implementation uses 32-bit word accumulation. Their equality tests the range proof’s intended arithmetic on real checkpoint values. Earlier unscoped projection measurements used

different resource limits and are retained separately. Their times are excluded from Table 4.

7 Conditional numerical bounds and token certificates

7.1 Projection error with rounded rescaling

The numerical development bounds the difference between the quantized and binary32 Lean recurrences on a common token sequence, with each recurrence retaining its own cache. A projection combines error already present in its input, fresh activation quantization error, weight reconstruction error, and rounding in both implementations. For reference coordinates x_i, w_i and reconstructed coordinates $\hat{x}_i, \hat{w}_i$ satisfying

$$|\hat{x}_i - x_i| \leq e_i^x, \quad |\hat{w}_i - w_i| \leq e_i^w,$$

the exact-product difference obeys

$$|\hat{x}_i \hat{w}_i - x_i w_i| \leq |w_i| e_i^x + |x_i| e_i^w + e_i^x e_i^w. \quad (13)$$

The incoming activation bound adds to the local reconstruction bound in e_i^x . This relation connects the quantized projection to the binary32 model’s current input, accounting for differences accumulated by earlier stages.

For an integer group accumulator a , the checked rescaling theorem bounds Equation (4) by

$$|u - a s^x s^w| \leq \varepsilon_{\text{out}} + |a| \varepsilon_{\text{scale}}. \quad (14)$$

The first term bounds the outer multiplication. The second accounts for the rounded scale product. The exact integer conversion follows from $|a| < 2^{24}$. The theorem requires finite scales and explicit bounds on both multiplication operands. Summing Equation (13) within a group, applying Equation (14), and adding the errors of ordered group additions and the final bias gives a component bound. The binary32 reference’s serial multiplication, addition, and bias errors also contribute.

The executable outward bound replaces per-coordinate magnitudes by checked uniform bounds. With $|x_i| \leq X$, $|w_i| \leq W$, $e_i^x \leq E_x$, and $e_i^w \leq E_w$, a group contribution is bounded by

$$\varepsilon_{\text{out}} + 1,032,256 \varepsilon_{\text{scale}} + 64(W E_x + X E_w + E_x E_w). \quad (15)$$

The projection bound multiplies the group-level expression and addition allowance by the number of groups, then includes bias and reference-computation rounding. The uniform grouped bound and paired projection theorem prove these steps. Repeated magnitude bounds and repeated propagation through all layers can make the bound large even when measured differences remain small.

7.2 Nonlinear operations and cached recurrence

LayerNorm comparison accounts for ordered mean and variance sums, centering, epsilon addition, square root, reciprocal, gamma multiplication, and beta addition. Its assumptions include finite inputs and parameters, arithmetic-range bounds, retained parameter equality, and positive lower bounds for both computed and real-reference denominators. The measured checker uses 1/1000 as the denominator lower bound. The smallest observed denominator among the original 229 positions is approximately 0.151385. A separate theorem supplies the real-reference root lower bound from the positive normalization epsilon.

Attention comparison follows query–key dot products, score scaling, maximum subtraction, exponential evaluation, softmax normalization, and probability–value products. It carries errors in

historical cached keys and values as well as errors in the current QKV projection. The exponential bound includes the degree-18 polynomial remainder, rounded coefficients, up to six halvings and corresponding squarings, and the cutoff below -64 . Exact reduction is an explicit premise. The soundness conversion also needs maximum dominance and positive denominator lower bounds. The retained softmax denominator bound is one.

The GELU theorem includes its argument arithmetic, the exponential computation, both quotient branches, and the magnitude-eight cutoff. It uses a conservative real tail bound of $1/100$ at that cutoff. Under the recorded range exponents, proved majorants bound the exponential error by $1/300$ and the complete GELU error formula by $1/16$. These are local approximation and rounding bounds under their stated premises. Residual additions contribute rounding from both recurrences and the error inherited from each input.

For each token, the cache bound retains the maximum of the prior cache error and the new key/value error. The hidden-state bound passes through all twelve transformer blocks, then through final normalization and the vocabulary projection. `Session.trace_error` proves a per-position, per-logit bound for the paired source traces. Its `Session.Conditions` predicate states the necessary arithmetic and reconstruction obligations against the source intermediates. Both initial caches can be empty, giving initial cache error zero.

The outward evaluator stores a nonnegative bound as a natural-number numerator divided by 2^{160} . Multiplication, division, and conversion from fractions round upward. Numerators have arbitrary precision. The outward arithmetic soundness proof and operation-specific proofs establish that this representation bounds the real error formulas. `ForwardUpper.trace_close` proves the complete executable bound recurrence, including both cache histories. Its premises include valid tokens and positions, correct cache shapes, successful quantized steps, finite magnitude bounds, local arithmetic bounds, reconstruction inequalities, and retained parameter equality.

7.3 Captured range evidence and forward failure

Native Lean checkers evaluate the finite range predicates on captured operands. Table 5 gives the coverage. Captures reproduce all recorded final logit hashes for both models. The normalization checker recomputes ordered sums. The GELU checker checks all Horner and squaring stages. The attention checker reconstructs the two cache histories and checks score and weighted-value arithmetic. Projection checks establish finite values and conservative ranges for dots, rescaling, group sums, and biases. Pointwise checks recompute captured embeddings and residual outputs bit for bit.

Checked records	Original 229 positions	Later 73 positions
LayerNorm calls, both models	11,450	3,650
GELU input words, both models	16,883,712	5,382,144
Attention calls, both models	5,496	1,752
Projection input/output records, both models	22,442	7,154
Paired embedding records	229	73
Paired residual-addition records	5,496	1,752

Table 5: Range-check coverage from the original and held-out evidence directories. Paired pointwise records contain the quantized and binary32 computations. Both sets use the same 49 checked projection matrices.

Captured operand identity remains a premise: the supplied operand arrays must equal the intermediates of the complete Lean recurrence. Matching final logit hashes supplies evidence for the capture, but it proves no internal operand equality in the kernel. The projection checker also

uses conservative arithmetic bounds without recomputing every full projection output. The native compiler and runtime evaluate the data predicates and bound numerators. The kernel checks the soundness implications from those predicates. These distinctions apply to the recorded numerical evaluation even though the separate binary execution theorem is complete.

The original forward evaluation and held-out forward evaluation produce a finite outward bound at every one of 302 evaluated positions. The smallest reported bound is approximately 1.516×10^{601} , and the largest is approximately 8.124×10^{61429} . Every bound exceeds 2^{129} , whereas the difference between two finite binary32 values is less than 2^{129} . Consequently, the evaluated forward bounds certify zero greedy choices and give no useful numerical-precision estimate. The exact integer numerators and evaluator identities remain in the evidence records.

Equation (15) and the nonlinear comparison formulas repeatedly replace component information by magnitude bounds and propagate those bounds through layers and cache history. They explain the form of the accumulated conservatism. The retained evaluation does not isolate each formula’s contribution to the final excess. Tighter bounds require analysis of those stages, with separate evidence for any proposed improvement.

7.4 Certificates from observed logit pairs

Let z^f, z^q be finite binary32 logit arrays, let k be the binary32 winner, and let c be a common offset. If bounds e_j satisfy

$$|z_j^q - c - z_j^f| \leq e_j \quad \text{and} \quad z_k^f - z_j^f > e_k + e_j \quad (j \neq k),$$

then $z_k^q > z_j^q$ for every competitor. The result follows by subtracting the two error inequalities. A common offset cancels from every pairwise difference. The checked logit predicate and soundness theorem implement the comparison with integers in units of 2^{-149} , so every finite binary32 input has an exact representation. They also check lengths, winner membership, and finiteness. Equality in the margin test is inconclusive. The general greedy operation retains the first index on ties, while an accepted strict certificate establishes a unique winner.

The raw certificate sets $c = 0$ and uses the observed component errors. The common-offset certificate sets $c = z_k^q - z_k^f$, which makes the winner’s error zero, then uses exact shifted component errors. Table 6 reports the two checks. The common-offset check certifies 232 observations. Of the 263 agreeing observations, 31 remain inconclusive under that sufficient condition. Certificate failure therefore cannot establish a changed winner.

Input set	Positions	Greedy agreement	Raw	Common offset
Fixed trace	128	120	85	116
Nine original prompt sequences	101	85	34	67
Six later prompt sequences	73	58	26	49
Total observations	302	263	145	232

Table 6: Certificates derived from observed logit pairs. Counts include overlapping prefixes between input sets. The forward-bound certificate count is zero for all three sets.

The packed-word theorem connects the predicate to returned byte arrays, and the cached-step theorem connects such arrays to the paired recurrences. A session theorem proves equal greedy choices along a common token sequence when every required step passes its certificate. The recorded acceptance counts establish individual choices for the supplied logit pairs. They use measured differences after both models have run. The numerical forward theorem would support a

different conclusion from propagated bounds, but its measured bounds satisfy no margin. Complete sampled-text identity or semantic correctness does not follow from either acceptance count.

8 Trust assumptions and remaining work

The execution result depends on the Lean kernel accepting the proof, the stated Lean axioms, and the adequacy of the formal decoder, verifier profile, integer and floating-point semantics, and Talos execution model for the claimed WebAssembly behavior. The proof connects those definitions to the embedded bytes. Native deployment further depends on the external-file check, runtime execution, host byte transfers and calls, operating system, and hardware. Arithmetic NaN canonicalization is enabled in the retained Wasmtime configuration. The package’s empty host-import assumption list concerns imported WebAssembly functions. It does not remove the external runtime and orchestration assumptions.

The numerical result adds assumptions about the input checkpoint relation, arithmetic ranges, positive denominators, magnitudes, source-intermediate identities, successful steps, and native checker evaluation. Source-level error theorems enter the quantized package’s declaration audit alongside the execution theorems. The principal exact-binary theorem in Equation (8) states execution behavior. Applying the numerical comparison to an observed deployment also requires the numerical premises and the connection from measured operands to the specified source computation.

The experiments compare one pinned model on a small fixed collection of inputs. Their strongest implementation test is exact agreement between the grouped WebAssembly and quantized reference across the retained complete caches and outputs. Their strongest numerical preservation result is the per-observation common-offset margin check. The generated texts record repetition and incorrect statements in both models. The runtime result describes a scalar CPU WebAssembly implementation on one host, with three measured full traces and a stated timing interval. It supplies no comparison with optimized native matrix libraries, SIMD quantized kernels, GPU inference, or other hardware.

The next numerical task is to tighten the forward bound and discharge captured-operand identity with the source recurrence through checked evidence. Componentwise bounds, better control of normalization and attention perturbations, and preservation of useful dependencies are possible directions whose benefit requires measurement and proof. A useful milestone would certify a token from pre-output error bounds and a strict winning margin. The present experiment establishes the theorem structure and a complete bound evaluation, with zero such certifications.

Repository release completion requires the remaining matcher proof repairs, current aggregate source and frozen-package checks, full execution and semantic-conformance runs, and a receipt from an immutable clean checkout. The quantized implementation’s binary identity and retained GPT evidence are already fixed. Later release receipts should identify their own source revision and input digest, so a repository-wide result cannot be inferred from the focused package’s success. The report’s source and data identifiers permit checking the reported component independently while those broader tasks continue.

A Source, artifact, and evidence identities

All repository links in this report refer to revision `c655d35b5011c1703dfd22bcceaec4e5bee17088` at <https://github.com/jsmorph/leanexe>. The proof workspace pins Lean 4.34.0-rc2, commit `6a10ac8c22beadecabdbb0919c2b50214762f91d`, and Talos revision `87e3aa5e8f6e6f3b3eb5e7e4c5aba43071002d47`. The exact-artifact verifier source identity is `0b8486bea82c65a31ecd332642f8d9`

6f9cf99f2726130f56707f373c02017194. The upstream model revision is 607a30d783dfa663caf39e06633721c8d4cfc7e. Table 7 fixes the model and binary files used by the measurements.

Table 7: SHA-256 identities.

Object	SHA-256
Group64 binary	9082c12c3b73aa6998a6d8ca0d97b509710e8a035afbf93587d80659ce773075
Binary32 binary	e93de126e00d7f5c5b9b30ca014a13b1385e9f91e3cb6b4e56a4aacf7a2b4ade
Group64 weight file	9d60657659e502b8dae9f11c2e73583643962b42c662cb9b51aa53e8730ec314
Binary32 weight file	6c12f993878ad39ba4aa3b0ab58a7466f5a62651cda94464dab438339008ba1d
Upstream safetensors	248dfc3911869ec493c76e65bf2fcf7f615828b0254c12b473182f0f81d3a707
Tokenizer JSON	8414cab924d8b9b33013f0d221c5862f365ee9be39c5c2bfae8a5a9e970478a6
Native session host	246419d68fc63c40049e09a10d8afc0e955ef3d4fe65043bf2713eed9b6d441
Original candidate binary	4fab215a51e58996b78dc3b988182eadcf47532f60556db7dd5d8d35468483ac
Original candidate weights	cdf0d8752b9b8243de911eab7435f4c96356752437d190ef4da094073f59c29a

A.1 Claim-to-declaration map

Table 8 identifies the statements supporting the proof claims. Names beginning with Q. abbreviate `Project.Gpt2QuantizedCached..` Names beginning with P. abbreviate `Project.ProofKit..` The linked modules contain the complete hypotheses and conclusions.

Table 8: Checked declarations and their scope.

Claim	Declaration and module
Integer sum equality and prefix range	<code>P.QuantizedDot.dot_exact</code> ; <code>P.QuantizedDot.dot_prefix_range</code>
Scalar reconstruction	<code>P.QuantizedScalarError.reconstruction</code>
Two rounded rescalings	<code>P.QuantizedRescaleError.rescale_error</code>
Complete token call	<code>Q.Entry.publicStep_exact</code> , exposed as <code>Q.Spec.cachedStep_exact</code>
Model validation	<code>Q.Validation.validateModel_exact</code>
Repeated calls and cleanup	<code>Q.Session.runs_exact</code> and <code>Q.Session.close_owned</code>
Reset through closed session	<code>Q.Spec.gpt2_128_exact</code>
Exact binary transfer	<code>Q.Artifact.artifact_gpt2_128_exact</code>
Checkpoint correspondence	<code>Q.Export.projection_checked</code> ; <code>Q.Export.retained_checked</code>

Claim	Declaration and module
Real session error formula	<code>Q.Numerical.Session.trace_error</code>
Outward session bound	<code>Q.Numerical.ForwardUpper.trace_close</code>
Forward margin implication	<code>Q.Numerical.ForwardUpper.choices_agree</code>
Observed-logit predicate	<code>P.F32LogitCertificate.sound</code> ; <code>P.F32LogitCertificate.greedy</code>
Packed and session certificates	<code>PackedLogitCertificate</code> , <code>LogitCertificates</code> , and <code>SessionCertificates</code>

A.2 Measurement records

The original evaluation manifest fixes the checkpoint, 128-token trace, nine prompt cases, and sampling parameters. The later manifest fixes the six additional prompts before their evaluation. Table 9 maps measured conclusions to retained records. Historical JSON status strings stating that a proof was pending record the state at capture time. The current proof status follows the declarations and package checks described in Section 3.

Table 9: Retained records under `data/gpt2-quantized-v1`.

Claim	Record
Storage and tensor layout	Grouped checkpoint manifest and current model record.
Original error source	Projection diagnostics, with local activation, weight, and combined errors.
Grouping and control	Fixed-prefix experiment and prompt-prefix experiment.
Bitwise model agreement	Grouped cached test.
Generated completions	Nine compiled cases and six later cases.
Runtime and memory	Warm full-model benchmark and scoped projection benchmark.
Checkpoint relation	Export check, including successful native evaluation and the earlier timeout.
Activation quantizers	Activation check, with exact quotient and reconstruction bounds.
Range predicates	Original range records and held-out range records, with checker logs, source digests, and compressed arithmetic profiles.
Observed-logit certificates	Original coverage and held-out coverage.
Forward-bound failure	Original outward evaluation and held-out outward evaluation, with compressed exact numerators.

A.3 Reproduction procedure

The repository’s development instructions fix the native and proof toolchains. The GPT-2 training and export instructions describe the checkpoint and Python environment. The quantized evaluation guide gives export, compile, reference-comparison, completion, and benchmark commands. All Lean, Lake, and compiler commands use `tools/leanrun` under the serialized resource limits. The exact-binary package can be checked with the following command, written over lines for readability:

```
tools/artifact-proof.js check
proofs/artifacts/gpt2_quantized_cached/
9082c12c3b73aa6998a6d8ca0d97b509710e8a035afbf93587d80659ce773075/
program.wasm
Project.Gpt2QuantizedCached.ArtifactTranslation
```

The directory and filename in that display form one path argument. The paper’s companion reproduction instructions preserve the executable command without line breaks inside arguments. The package manifest fixes the embedded-byte definition, decoder and validator theorems, translated program, behavioral declarations, and verifier identity. The focused source check uses `tools/talos-proof.js check gpt2_quantized_cached`.

Export with `quantized.py export-model --scheme group64` recreates the scheme-two weight file. The cached test, text comparison, and benchmark scripts accept `--grouped` and an explicit `--wasm` path. The recorded benchmark uses `--repetitions 3`. Original and later certificate records preserve the capture commands, native checker commands, source digests, data digests, and output logs. Raw captures generated under `build` are reproducible from those scripts. Compressed profiles and forward-bound numerators remain in the tracked evidence directory.

The companion `evidence/inventory.json` records the content hashes of every source and data file selected for the report. Its summary script recomputes the reported agreement counts, certificate counts, storage reduction, and runtime ratio from the retained JSON. These checks validate the report’s use of evidence. Rechecking Lean theorems or rerunning inference uses the repository commands above and remains a separate operation.

References

- [1] Codex GPT-6 and Jamie Stephens. Exact execution verification of cached GPT-2 in Lean and WebAssembly, 2026. marXiv:2609.00011. <http://127.0.0.1:8405/abs/2609.00011>.
- [2] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, volume 35, 2022. <https://arxiv.org/abs/2208.07339>.
- [3] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018. <https://arxiv.org/abs/1712.05877>.
- [4] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. <https://cdn.openai.com/better-language-models/language-models.pdf>.

- [5] Jamie Stephens. GPT-2 inference from Lean to WebAssembly and WGSL: Artifact verification, shader proofs, and execution evidence, 2026. marXiv:2609.00014. <http://127.0.0.1:8405/abs/2609.00014>.
- [6] WebAssembly Community Group. WebAssembly Core Specification: Numerics, 2026. Accessed 24 September 2026. <https://webassembly.github.io/spec/core/exec/numerics.html>.
- [7] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, 2023. <https://arxiv.org/abs/2211.10438>.
- [8] Yedi Zhang, Fu Song, and Jun Sun. QEBVerif: Quantization error bound verification of neural networks. In *Computer Aided Verification*, volume 13965 of *Lecture Notes in Computer Science*, pages 413–437, 2023. <https://arxiv.org/abs/2212.02781>.